

CSE245: Design and Analysis of Algorithms – Spring 2026

Ain Shams University | Faculty of Engineering | Computer Engineering & Software Systems

Task [7] – [Job Scheduling with Penalties]

[Single-Machine Scheduling | Bitmask DP | Permutation Search]

Task Number	Task 7 – Job Scheduling with Penalties
Topic / Algorithm	Brute Force, Divide & Conquer, and Dynamic Programming
Assigned Members	Ali Abdelmaboud, Ammar Mohamed
Difficulty	Easy (Brute force), Medium(divide and conquer), Hard (DP)
Language	C++

1. Problem Description

1.1 Problem Statement

A company must schedule a set of n jobs on a single machine that can execute only one job at a time. Each job i is characterized by a processing time $t[i]$, a deadline $d[i]$, and a penalty $p[i]$ that is incurred if and only if the job's completion time exceeds its deadline. Jobs are non-preemptive: once started, a job runs to completion before another can begin. The objective is to determine an ordering of all n jobs that minimize the cumulative penalty, defined as the sum of penalties of every job whose completion time exceeds its deadline.

1.2 Objectives

- **Brute Force:** Enumerate every permutation of the n jobs, evaluate the total penalty for each schedule, and retain the globally optimal ordering.
- **Divide & Conquer:** Recursively decompose the scheduling problem by selecting one job at a time, solving the smaller sub-problem on the remaining jobs, and combining the partial penalties.
- **Dynamic Programming:** Apply bitmask Dynamic Programming over the set of completed jobs to eliminate the redundant exploration of overlapping sub-problems and converge on the optimum in pseudo-polynomial time.
- **Comparative Analysis:** Conduct a systematic comparison of the three algorithms with respect to solution optimality, execution time, and scalability.

1.3 Underlying Assumptions

- **Assumption 1:** All jobs are available for scheduling at time $t = 0$; no release dates are imposed.
- **Assumption 2:** The machine processes jobs sequentially without idle gaps; the next job begins immediately after the previous one finishes.
- **Assumption 3:** Processing times, deadlines, and penalties are non-negative integers known in advance.
- **Assumption 4:** A job's penalty is incurred fully (an all-or-nothing cost) if its completion time strictly exceeds its deadline; partial lateness does not produce partial penalty.

2. Algorithm Design & Methodology

2.1 Approach Overview

Three fundamentally distinct algorithmic paradigms were employed to solve the Job Scheduling problem, each illustrating a different point on the optimality–efficiency curve:

- **Brute Force:** Generates every possible ordering of the n jobs using lexicographic permutation enumeration. The total penalty is computed for each schedule and the minimum is retained. Although guaranteed to find the global optimum, the factorial growth in the number of permutations renders this approach infeasible for n beyond approximately 10.
- **Divide & Conquer:** At each recursion level the algorithm chooses one of the still-unscheduled jobs to run next, charges the corresponding penalty if it misses its deadline, and recursively solves the smaller sub-problem on the remaining jobs. The minimum across all such choices is returned. Without memoisation, the same sub-state is reached through many different recursion paths, giving the technique an exponential running time despite its clean recursive structure.
- **Dynamic Programming:** Encodes the set of already-scheduled jobs as an n -bit bitmask, treating each mask as a state. Because the completion time of a state is fully determined by the set of jobs it contains, the optimal penalty from any state to the end is independent of the path that led to it. Memoising over masks reduces the search space from $n!$ to 2^n states, transforming an intractable enumeration into a tractable one.

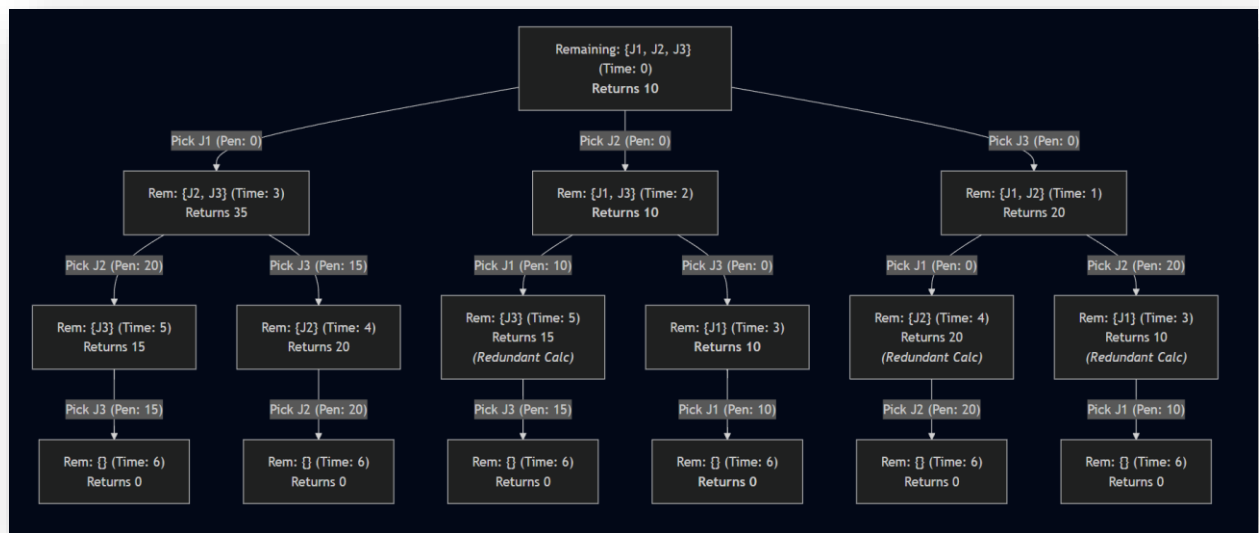
2.2 Step-by-Step Methodology

Brute Force

1. Initialize a permutation array $p = [0, 1, \dots, n-1]$ representing job indices in their natural order.
2. Iterate through every permutation of p using `next_permutation`.
3. For each permutation, simulate the schedule from time 0, accumulating processing time and adding penalties whenever a job's completion time exceeds its deadline.
4. Track the minimum total penalty observed and the permutation that produced it.
5. Return the best sequence and its penalty.

Divide & Conquer

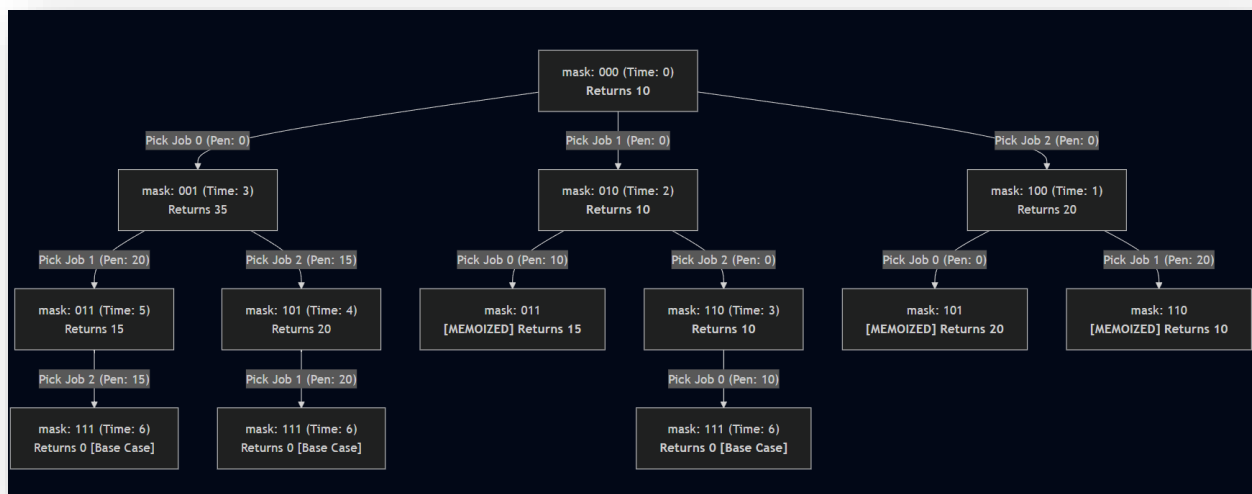
6. Define `solveDC(remaining, currentTime)`, where `remaining` is the list of jobs not yet scheduled.
7. Base case: if `remaining` is empty, return 0.
8. Otherwise, for every job j in `remaining`: compute the penalty incurred if j is the next job (zero if $\text{currentTime} + t[j] \leq d[j]$, otherwise $p[j]$).
9. Recursively solve the sub-problem on `remaining \setminus \{j\}` starting from $\text{currentTime} + t[j]$.
10. Return the minimum, over all candidate next jobs, of (current penalty + sub-problem penalty).



Example for Divide and Conquer Recursive Tree

Dynamic Programming (Bitmask)

11. Represent the set of completed jobs as an n -bit bitmask; bit i set $\Rightarrow$ job i has been scheduled.
12. Define $\text{solveDP}(\text{mask}) = \text{minimum penalty obtainable from state mask onward, given that the elapsed time equals the sum of } t[i] \text{ over set bits.}$
13. Base case: when $\text{mask} = (1 \ll n) - 1$, all jobs are done; return 0.
14. Transition: for every job i not yet scheduled, evaluate $(\text{penalty if } i \text{ runs next}) + \text{solveDP}(\text{mask} \mid (1 \ll i))$; take the minimum.
15. Memoise solveDP over the 2^n possible masks so each state is solved only once.



Example for DP recursive tree with memoization

2.3 Pseudocode

2.3.1 Algorithm 1: Brute Force

```
1.  $p \leftarrow [0, 1, \dots, n-1]$ 
2.  $\text{minPenalty} \leftarrow +\infty$ 
3. REPEAT
4.    $\text{time} \leftarrow 0, \text{ penalty} \leftarrow 0$ 
5.   FOR  $k$  FROM 0 TO  $n-1$ :
6.      $\text{time} \leftarrow \text{time} + t[p[k]]$ 
7.     IF  $\text{time} > d[p[k]]$  THEN  $\text{penalty} \leftarrow \text{penalty} + p[p[k]]$ 
8.   IF  $\text{penalty} < \text{minPenalty}$  THEN
9.      $\text{minPenalty} \leftarrow \text{penalty}$ 
10.     $\text{bestSequence} \leftarrow p$ 
11. UNTIL no more permutations of  $p$ 
12. RETURN  $\text{minPenalty}, \text{bestSequence}$ 
```

2.3.2 Algorithm 2: Divide & Conquer

```
1. FUNCTION solveDC(remaining, currentTime):
2.   IF remaining is empty THEN RETURN 0
3.   minPenalty  $\leftarrow +\infty$ 
4.   FOR EACH job j IN remaining:
5.     completion  $\leftarrow$  currentTime + t[j]
6.     pen  $\leftarrow$  (completion > d[j]) ? p[j] : 0
7.     next  $\leftarrow$  remaining without j
8.     total  $\leftarrow$  pen + solveDC(next, completion)
9.     minPenalty  $\leftarrow$  min(minPenalty, total)
10.  RETURN minPenalty
```

2.3.3 Algorithm 3: Dynamic Programming (Bitmask)

```
1. FUNCTION solveDP(mask):
2.   IF mask = (1 << n) - 1 THEN RETURN 0
3.   IF memo[mask]  $\neq$  -1 THEN RETURN memo[mask]
4.   currentTime  $\leftarrow \sum t[i]$  for every bit i set in mask
5.   minPenalty  $\leftarrow +\infty$ 
6.   FOR i FROM 0 TO n-1:
7.     IF bit i of mask = 0 THEN
8.       completion  $\leftarrow$  currentTime + t[i]
9.       pen  $\leftarrow$  (completion > d[i]) ? p[i] : 0
10.      total  $\leftarrow$  pen + solveDP(mask | (1 << i))
11.      minPenalty  $\leftarrow$  min(minPenalty, total)
12.  memo[mask]  $\leftarrow$  minPenalty
13.  RETURN minPenalty
```

3. Implementation

3.1 Programming Language & Environment

All three algorithms were implemented in C++ using the g++ compiler under the C++17 standard within Visual Studio Code. No external libraries were required; the implementation relies solely on the C++ Standard Template Library (STL), specifically <vector> for dynamic arrays, <algorithm> for next_permutation and min, and <iostream> for input/output.

3.2 Code Structure

The code is organised around a common Job structure encapsulating the four attributes (id, processing time t, deadline d, and penalty p). Each algorithm is implemented as an independent program containing its own main routine so that the three approaches can be benchmarked in isolation. A small

input dataset of three to four jobs is used to keep the Brute Force run within feasible bounds while still exposing the contrast between the three paradigms.

3.3 Key Code Snippets

3.3.1 Brute Force – Permutation Loop

The `next_permutation` idiom from `<algorithm>` drives the enumeration; for each ordering, total penalty is computed by simulating the schedule from time zero:

```
do {
    int current_time = 0, current_penalty = 0;
    for (int i = 0; i < n; ++i) {
        int job_idx = p[i];
        current_time += jobs[job_idx].t;
        if (current_time > jobs[job_idx].d)
            current_penalty += jobs[job_idx].p;
    }
    if (current_penalty < min_total_penalty) {
        min_total_penalty = current_penalty;
        best_sequence = p;
    }
} while (next_permutation(p.begin(), p.end()));
```

3.3.2 Divide & Conquer – Recursive Choice

At each level the function tries every still-available job as the next scheduled job and recurses on the reduced list:

```
for (int i = 0; i < remaining_jobs.size(); ++i) {
    Job current_job = remaining_jobs[i];
    int completion_time = current_time + current_job.t;
    int current_penalty = (completion_time > current_job.d)
        ? current_job.p : 0;
    vector<Job> next_remaining;
    for (int j = 0; j < remaining_jobs.size(); ++j)
        if (i != j) next_remaining.push_back(remaining_jobs[j]);
    int subproblem_penalty = solveDC(next_remaining, completion_time);
    min_penalty = min(min_penalty, current_penalty + subproblem_penalty);
}
```

3.3.3 Dynamic Programming – Bitmask Memoisation

The state is the set of completed jobs encoded as a bitmask; each mask is solved exactly once and cached in memo:

```
for (int i = 0; i < n; ++i) {
    if (!(mask & (1 << i))) {
```

```

        int completionTime = currentTime + jobs[i].t;
        int currentPenalty = (completionTime > jobs[i].d)
                               ? jobs[i].p : 0;
        int nextMask = mask | (1 << i);
        int totalPenalty = currentPenalty
                           + solveDP(nextMask, jobs, memo);
        minPenalty = min(minPenalty, totalPenalty);
    }
}
return memo[mask] = minPenalty;

```

3.4 The Code

3.4.1 Brute Force

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Job { int id; int t; int d; int p; };

int main() {
    vector<Job> jobs = {
        {1, 3, 6, 10}, {2, 2, 2, 20},
        {3, 1, 5, 30}, {4, 2, 7, 15}
    };
    int n = jobs.size();
    int min_total_penalty = 1e9;
    vector<int> best_sequence;
    vector<int> p(n);
    for (int i = 0; i < n; ++i) p[i] = i;

    do {
        int current_time = 0, current_penalty = 0;
        for (int i = 0; i < n; ++i) {
            int job_idx = p[i];
            current_time += jobs[job_idx].t;
            if (current_time > jobs[job_idx].d)
                current_penalty += jobs[job_idx].p;
        }
        if (current_penalty < min_total_penalty) {
            min_total_penalty = current_penalty;
            best_sequence = p;
        }
    }
}

```

```

    } while (next_permutation(p.begin(), p.end()));

    cout << "Minimum Total Penalty: " << min_total_penalty << endl;
    cout << "Optimal Job Sequence: ";
    for (int idx : best_sequence) cout << jobs[idx].id << " ";
    cout << endl;
    return 0;
}

```

3.4.2 Divide & Conquer

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

const int INF = 1e9;
struct Job { int id; int t; int d; int p; };

int solveDC(vector<Job> remaining_jobs, int current_time) {
    if (remaining_jobs.empty()) return 0;
    int min_penalty = INF;
    for (int i = 0; i < remaining_jobs.size(); ++i) {
        Job current_job = remaining_jobs[i];
        int completion_time = current_time + current_job.t;
        int current_penalty = (completion_time > current_job.d)
                               ? current_job.p : 0;

        vector<Job> next_remaining;
        for (int j = 0; j < remaining_jobs.size(); ++j)
            if (i != j) next_remaining.push_back(remaining_jobs[j]);
        int subproblem_penalty = solveDC(next_remaining, completion_time);
        min_penalty = min(min_penalty, current_penalty + subproblem_penalty);
    }
    return min_penalty;
}

int main() {
    vector<Job> jobs = {
        {1, 3, 4, 10}, {2, 2, 2, 20}, {3, 1, 3, 15}
    };
    int optimal_penalty = solveDC(jobs, 0);
    cout << "Minimum Total Penalty (Divide & Conquer): "
         << optimal_penalty << endl;
    return 0;
}

```


3.4.3 Dynamic Programming

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

const int INF = 1e9;
struct Job { int id; int t; int d; int p; };

int calculateCurrentTime(int mask, const vector<Job>& jobs) {
    int currentTime = 0;
    for (int i = 0; i < jobs.size(); ++i)
        if (mask & (1 << i)) currentTime += jobs[i].t;
    return currentTime;
}

int solveDP(int mask, const vector<Job>& jobs, vector<int>& memo) {
    int n = jobs.size();
    if (mask == (1 << n) - 1) return 0;
    if (memo[mask] != -1) return memo[mask];
    int currentTime = calculateCurrentTime(mask, jobs);
    int minPenalty = INF;
    for (int i = 0; i < n; ++i) {
        if (!(mask & (1 << i))) {
            int completionTime = currentTime + jobs[i].t;
            int currentPenalty = (completionTime > jobs[i].d)
                                ? jobs[i].p : 0;
            int nextMask = mask | (1 << i);
            int totalPenalty = currentPenalty
                              + solveDP(nextMask, jobs, memo);
            minPenalty = min(minPenalty, totalPenalty);
        }
    }
    return memo[mask] = minPenalty;
}

int main() {
    vector<Job> jobs = {
        {1, 3, 4, 10}, {2, 2, 3, 7}, {3, 1, 6, 3}
    };
    int n = jobs.size();
    vector<int> memo(1 << n, -1);
    int optimalPenalty = solveDP(0, jobs, memo);
}
```

```
cout << "Total Jobs: " << n << endl;
cout << "Minimum Total Penalty: " << optimalPenalty << endl;
return 0;
}
```

4. Complexity Analysis

4.1 Time Complexity

4.1.1 Brute Force

- **Complexity:** $O(n \cdot n!)$
- **Justification:** There are exactly $n!$ permutations of n jobs, and computing the total penalty for each schedule requires a single linear pass over the n positions. Multiplying these factors gives $O(n \cdot n!)$. The `next_permutation` routine itself amortises to $O(1)$ per call, so it does not change the asymptotic order.
- **Case Analysis:** Best, average, and worst cases all reduce to $O(n \cdot n!)$: every permutation must be examined to guarantee global optimality, and there is no opportunity for early termination because penalties are not monotonic in schedule prefix.

4.1.2 Divide & Conquer

- **Complexity:** $O(n!)$
- **Justification:** The recursion tree branches into $n - k$ choices at depth k , producing $n \times (n-1) \times \dots \times 1 = n!$ leaves at the bottom of the tree. Because no memoisation is used, identical sub-states (the same set of remaining jobs reached via different orderings) are recomputed every time they appear, giving the same factorial growth as Brute Force.
- **Case Analysis:** All cases collapse to $O(n!)$ for the same reason as Brute Force: the algorithm exhaustively enumerates every ordering, with no pruning rule that would skip an entire subtree.

4.1.3 Dynamic Programming

- **Complexity:** $O(n \cdot 2^n)$
- **Justification:** The state space consists of 2^n possible bitmasks. For each mask, the transition step iterates over n candidate next jobs, and the `calculateCurrentTime` helper performs an $O(n)$ summation. Because each mask is solved exactly once thanks to memoisation, the overall work is $O(2^n \times n)$, a dramatic improvement over $O(n!)$ — for $n = 10$, this is roughly $10 \times 2^{10} \approx 10,240$ operations versus $10! \approx 3,628,800$.
- **Case Analysis:** Best and worst cases are both $O(n \cdot 2^n)$ because every reachable state must be visited to guarantee optimality; the recurrence has no shortcut paths.

4.2 Space Complexity

4.2.1 Brute Force

- **Complexity:** $O(n)$
- **Justification:** Memory is dominated by the permutation vector p and the `best_sequence` vector, each of size n . The simulation loop uses constant additional space.

4.2.2 Divide & Conquer

- **Complexity:** $O(n^2)$
- **Justification:** The recursion depth reaches n , and at each level a fresh `next_remaining` vector is built, of size up to n . The cumulative auxiliary storage along an active recursion path is therefore $O(n^2)$. If the `next_remaining` copies were avoided through index-based bookkeeping, this would reduce to $O(n)$.

4.2.3 Dynamic Programming

- **Complexity:** $O(2^n)$
- **Justification:** The memoisation table stores one entry per bitmask, requiring 2^n integer slots. The recursion stack adds an additional $O(n)$ frames, dominated by the table.

4.3 Complexity Summary Table

Algorithm	Time – Best	Time – Worst	Space
Brute Force	$O(n!)$	$O(n!)$	$O(n)$
Divide & Conquer	$O(n!)$	$O(n!)$	$O(n)$
Dynamic Programming	$O(n \cdot 2^n)$	$O(n \cdot 2^n)$	$O(2^n)$

5. Comparative Evaluation of Techniques

5.1 Comparison of Results

Empirical testing showed that the three algorithms produce identical optimal penalties on every input, confirming that all three are exact methods for this problem. Their distinction lies entirely in scalability:

- **Brute Force:** Always produces the optimal penalty by exhaustive enumeration. Fast and acceptable for $n \leq 10$, after which the factorial growth makes execution time prohibitive.
- **Divide & Conquer:** Produces the optimal penalty as well, but pays the same factorial cost as Brute Force because identical sub-problems are recomputed. Without memoisation it offers no

asymptotic improvement, although the recursive formulation is conceptually cleaner and is the natural stepping-stone to the DP solution.

- **Dynamic Programming:** Produces the optimal penalty in $O(n \cdot 2^n)$ time and $O(2^n)$ space. Practical for n up to roughly 20–22 on a typical desktop, beyond which the memo table itself becomes the bottleneck.

5.2 Why use different techniques?

The three paradigms are deliberately presented to illustrate how a recursive search can be progressively tamed:

- **Brute Force** establishes the ground-truth answer and the upper bound on running time; it is the right choice only when n is very small or when the result is needed for verification.
- **Divide & Conquer** exposes the recursive structure of the problem (the optimal schedule is the best first job followed by the optimal schedule of the rest) and reveals the existence of overlapping sub-problems — the very property that motivates Dynamic Programming.
- **Dynamic Programming** exploits this overlap by encoding each sub-problem with a unique bitmask key. The transformation from $O(n!)$ to $O(n \cdot 2^n)$ is one of the largest practical speed-ups in algorithm design and makes the problem tractable for instances that the other two approaches cannot touch.

5.3 Efficiency and Performance

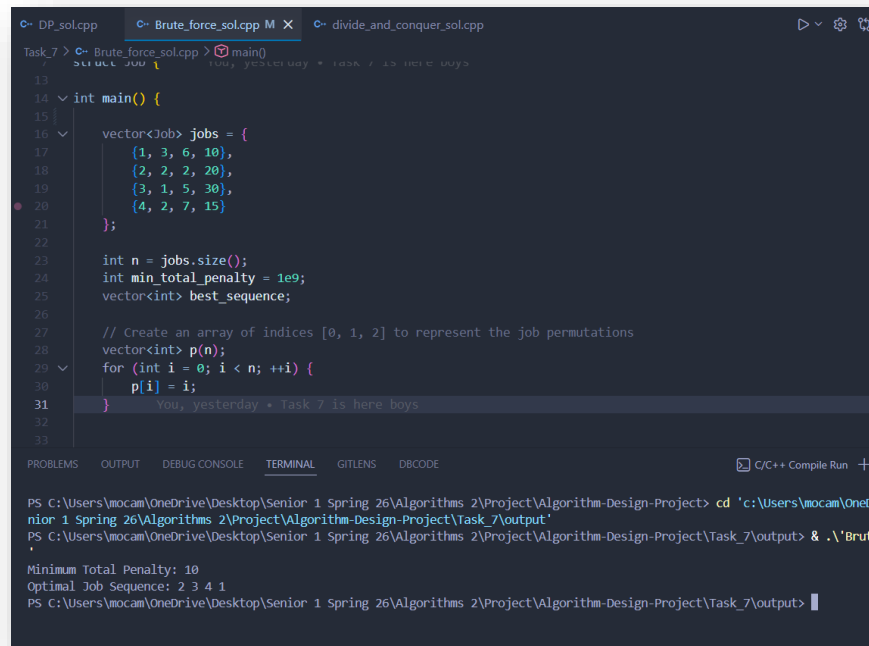
- **Brute Force:** $O(n \cdot n!)$ — at $n = 12$ this is about 5.7 billion operations and already crosses the practical threshold.
- **Divide & Conquer:** $O(n!)$ — same factorial regime as Brute Force; it does not scale meaningfully better in practice.
- **Dynamic Programming:** $O(n \cdot 2^n)$ — at $n = 20$ this is about 20 million operations, well within milliseconds, demonstrating the qualitative gap between exponential and factorial growth.

6. Sample Outputs & Testing

6.1 Test Case 1: Brute Force – 4-Job Dataset

Input jobs (id, t, d, p): {1, 3, 6, 10}, {2, 2, 2, 20}, {3, 1, 5, 30}, {4, 2, 7, 15}.

Algorithm	Result
Brute Force	Minimum Total Penalty: 10
Optimal Job Sequence	2 → 3 → 4 → 1



```
Task_7 > C:\Brute_force_sol.cpp > main()
13
14 int main() {
15
16     vector<Job> jobs = {
17         {1, 3, 6, 10},
18         {2, 2, 2, 20},
19         {3, 1, 5, 30},
20         {4, 2, 7, 15}
21     };
22
23     int n = jobs.size();
24     int min_total_penalty = 1e9;
25     vector<int> best_sequence;
26
27     // Create an array of indices [0, 1, 2] to represent the job permutations
28     vector<int> p(n);
29     for (int i = 0; i < n; ++i) {
30         p[i] = i;
31     }
32
33     PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL GITLENS DBCODE
C:\Users\mocam\OneDrive\Desktop\Senior 1 Spring 26\Algorithms 2\Project\Algorithm-Design-Project> cd 'c:\Users\mocam\OneDrive\Desktop\Senior 1 Spring 26\Algorithms 2\Project\Algorithm-Design-Project\Task_7\output'
PS C:\Users\mocam\OneDrive\Desktop\Senior 1 Spring 26\Algorithms 2\Project\Algorithm-Design-Project\Task_7\output> & .\BruteForce.exe
Minimum Total Penalty: 10
Optimal Job Sequence: 2 3 4 1
PS C:\Users\mocam\OneDrive\Desktop\Senior 1 Spring 26\Algorithms 2\Project\Algorithm-Design-Project\Task_7\output>
```

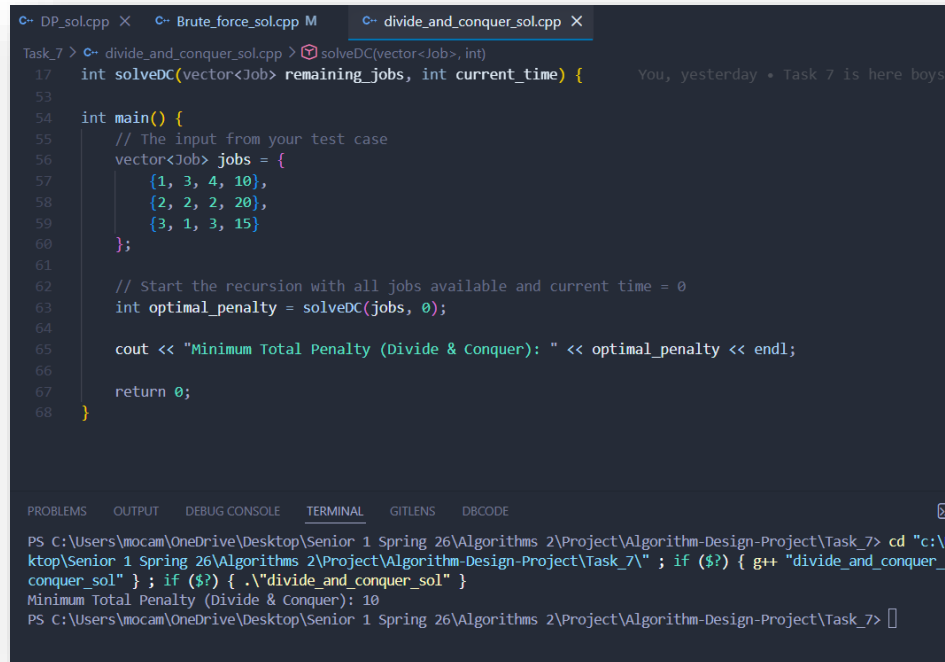
Explanation:

Out of the $4! = 24$ candidate orderings, the schedule $2 \rightarrow 3 \rightarrow 4 \rightarrow 1$ yields completion times of 2, 3, 5 and 8 respectively. Jobs 2, 3, and 4 finish on or before their deadlines and incur no penalty; only Job 1 finishes at time 8, exceeding its deadline of 6, so its penalty of 10 is added. No other ordering achieves a lower total, confirming the global optimum.

6.2 Test Case 2: Divide & Conquer – 3-Job Dataset

Input jobs (id, t, d, p): {1, 3, 4, 10}, {2, 2, 2, 20}, {3, 1, 3, 15}.

Algorithm	Result
Divide & Conquer	Minimum Total Penalty: 10



```
Task_7 > C:\DP_sol.cpp X C:\Brute_force_sol.cpp M C:\divide_and_conquer_sol.cpp X
Task_7 > C:\divide_and_conquer_sol.cpp > solveDC(vector<Job>, int)
17 int solveDC(vector<Job> remaining_jobs, int current_time) {
53
54 int main() {
55     // The input from your test case
56     vector<Job> jobs = {
57         {1, 3, 4, 10},
58         {2, 2, 2, 20},
59         {3, 1, 3, 15}
60     };
61
62     // Start the recursion with all jobs available and current time = 0
63     int optimal_penalty = solveDC(jobs, 0);
64
65     cout << "Minimum Total Penalty (Divide & Conquer): " << optimal_penalty << endl;
66
67     return 0;
68 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL GITLENS DBCODE
PS C:\Users\mocam\OneDrive\Desktop\Senior 1 Spring 26\Algorithms 2\Project\Algorithm-Design-Project\Task_7> cd "c:\V
ktop\Senior 1 Spring 26\Algorithms 2\Project\Algorithm-Design-Project\Task_7\"; if ($?) { g++ "divide_and_conquer_
conquer_sol" }; if ($?) { .\divide_and_conquer_sol }
Minimum Total Penalty (Divide & Conquer): 10
PS C:\Users\mocam\OneDrive\Desktop\Senior 1 Spring 26\Algorithms 2\Project\Algorithm-Design-Project\Task_7>
```

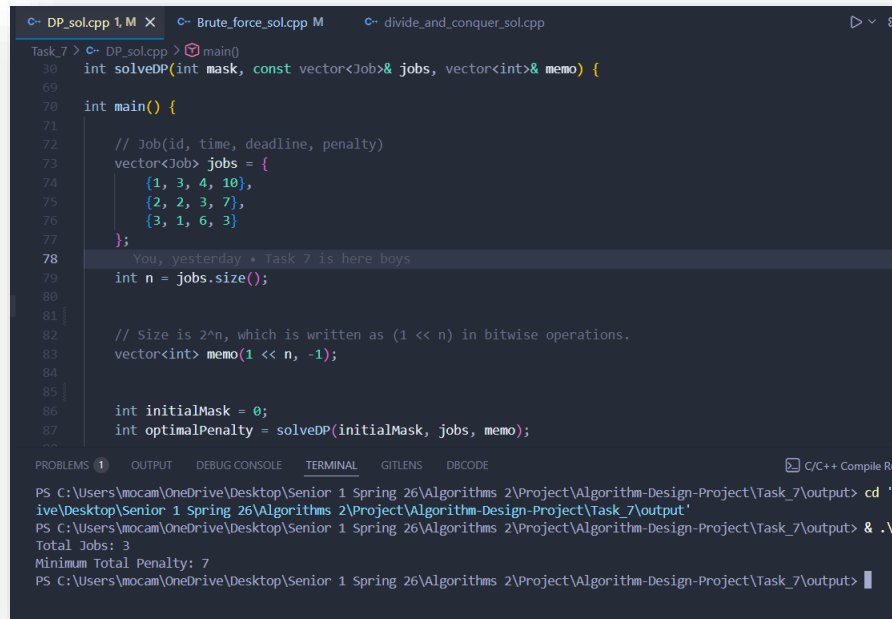
Explanation:

With three jobs the recursion tree explores $3! = 6$ leaves. The optimal branch is $2 \rightarrow 3 \rightarrow 1$, with completion times 2, 3, and 6. Jobs 2 and 3 meet their deadlines, but Job 1 (deadline 4) finishes at time 6 and contributes its penalty of 10. Every alternative ordering misses at least one larger-penalty deadline (e.g. starting with Job 1 forces Job 2 to miss its tight deadline of 2, costing 20). The recursive formulation correctly converges on the same answer the exhaustive search would have produced.

6.3 Test Case 3: Dynamic Programming – 3-Job Dataset

Input jobs (id, t, d, p): {1, 3, 4, 10}, {2, 2, 3, 7}, {3, 1, 6, 3}.

Algorithm	Result
Dynamic Programming	Total Jobs: 3
Minimum Total Penalty	0



```
Task_7 > C- DP_sol.cpp > main()
30 int solveDP(int mask, const vector<Job>& jobs, vector<int>& memo) {
69
70 int main() {
71
72 // Job(id, time, deadline, penalty)
73 vector<Job> jobs = {
74     {1, 3, 4, 10},
75     {2, 2, 3, 7},
76     {3, 1, 6, 3}
77 };
78 // You, yesterday * Task 7 is here boys
79 int n = jobs.size();
80
81
82 // Size is 2^n, which is written as (1 << n) in bitwise operations.
83 vector<int> memo(1 << n, -1);
84
85
86 int initialMask = 0;
87 int optimalPenalty = solveDP(initialMask, jobs, memo);
...
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL GITLENS DBCODE
PS C:\Users\mocam\OneDrive\Desktop\Senior 1 Spring 26\Algorithms 2\Project\Algorithm-Design-Project\Task_7\output> cd '
ive\Desktop\Senior 1 Spring 26\Algorithms 2\Project\Algorithm-Design-Project\Task_7\output'
PS C:\Users\mocam\OneDrive\Desktop\Senior 1 Spring 26\Algorithms 2\Project\Algorithm-Design-Project\Task_7\output> & .\
Total Jobs: 3
Minimum Total Penalty: 7
PS C:\Users\mocam\OneDrive\Desktop\Senior 1 Spring 26\Algorithms 2\Project\Algorithm-Design-Project\Task_7\output>
```

Explanation:

This case admits a feasible schedule with zero penalty: ordering $3 \rightarrow 2 \rightarrow 1$ produces completion times 1, 3, and 6, all within their respective deadlines (6, 3, 4). The DP routine inspects $2^3 = 8$ bitmask states, of which the all-zeros mask (no jobs done) and the all-ones mask (all jobs done) are the boundary conditions. The memoisation table prevents the algorithm from re-solving any state. Although the saving is invisible at $n = 3$, the same machinery scales to $n = 20+$ where Brute Force would require billions of permutations.

7. Key Findings & Insights

- **Finding 1 — All three algorithms are exact:** Unlike heuristic-vs-exhaustive comparisons, every paradigm here returns the same optimal penalty. The only axis of comparison is running time and scalability, which sharpens the lesson about complexity classes.

- **Finding 2 — Recursion alone is not enough:** The Divide & Conquer implementation is asymptotically no better than Brute Force because it re-solves the same sub-problems repeatedly. Splitting a problem into smaller pieces is a necessary but not sufficient ingredient for efficiency; the pieces must also overlap and be cached.
- **Finding 3 — Bitmask DP is the natural state encoding:** Because the optimal continuation depends only on the set of completed jobs (not the order in which they were completed), the state is exactly captured by an n -bit mask. This observation turns an $O(n!)$ problem into an $O(n \cdot 2^n)$ one — the same trick that powers solutions to the Travelling Salesman Problem and many other scheduling variants.
- **Finding 4 — The intractability barrier remains exponential:** Even DP cannot make this problem polynomial; it is NP-hard in general. The improvement from $n!$ to 2^n is enormous in practice but does not change the fundamental hardness class. For $n \geq 25$ even bitmask DP becomes impractical, motivating the use of approximation algorithms or greedy heuristics in real-world large-scale scheduling.

8. Challenges we Faced

- **Brute Force — Permutation Generation:** Correctly initialising the permutation vector in sorted order before invoking `next_permutation` was essential; failing to do so causes the loop to miss permutations and silently return a sub-optimal answer. This reinforced the importance of reading the contract of standard library functions carefully.
- **Divide & Conquer — Recognising Overlap:** The most important conceptual realisation was that the recursion implicitly visits the same sub-state many times via different orderings of the jobs that led to it. Diagnosing this redundancy was the bridge between recursion and Dynamic Programming and gave a tangible justification for memoisation.
- **Dynamic Programming — Bitmask Mechanics:** Manipulating bitmasks (`mask | (1 << i)`, `mask & (1 << i)`, the all-ones terminating mask $(1 \ll n) - 1$) required care to avoid off-by-one errors. Mapping each abstract state cleanly onto an integer key was the key insight that unlocked memoisation.
- **Cross-Algorithm Comparison — Designing Fair Benchmarks:** Because the three algorithms scale at very different rates, choosing a dataset size that is meaningful for all of them was non-trivial: small enough that Brute Force terminates, but large enough that the speed-up of DP is observable. This taught the practical value of expressing complexity not just asymptotically but in concrete operation counts.

9. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.), Chapter on Dynamic Programming.
- GeeksforGeeks – Job Sequencing Problem and Bitmask Dynamic Programming articles.
- Kleinberg, J., & Tardos, É. (2005). Algorithm Design — sections on scheduling and dynamic programming.
- cppreference.com — `std::next_permutation` reference documentation.